

EXAM POV NOTES (Q&A FORMAT)**Q1: What is the Transport Layer, and what are its key roles and services in computer networks?**
(10 Marks / 10 Points)

The Transport Layer is the fourth layer in the standard network model, acting as a smart, organized layer that decides how data should travel based on the application's needs. Its primary role is to provide logical, direct communication services to specific application processes running on different hosts, rather than just between two computers.

- **Logical Communication:** It provides a logical communication line between application processes on different hosts, enabling them to send messages to each other even without a physical connection. This distinction is crucial, as it allows, for example, your Chrome browser to communicate specifically with web server software.
- **Implementation Location:** This entire operation, including the implementation of transport layer protocols, lives inside your device, known as the end system. It is not handled by network routers.
- **Key Services Categories:** The services provided by transport layer protocols can be broadly divided into categories such as end-to-end delivery, reliable delivery, flow control, multiplexing/demultiplexing, and addressing. All transport layer protocols provide multiplexing/demultiplexing service.
- **End-to-End Delivery:** It ensures the transmission of the entire message from the source to the destination, guaranteeing end-to-end delivery. Other parts of the network might check for errors in short hops, but the Transport Layer is the star for end-to-end verification.
- **Reliable Delivery:** This service provides reliability by retransmitting lost and damaged packets, aiming for error-free transmission across the entire journey. It focuses on four pillars: error control, sequence control, loss control, and duplication control.
- **Error Control:** It's about finding and fixing any data that got scrambled along the way, performing an end-to-end check to verify data across the entire journey. This is more comprehensive than the data link layer's node-to-node error checks, which would miss errors inside a router.
- **Sequence Control:** Data is often chopped into tiny packets that can arrive out of order, so this pillar is crucial for putting them all back together in the right sequence at the receiving end. On the sending end, it ensures packets from upper layers are properly prepared for lower layers.
- **Loss Control:** This aspect ensures that all fragments of a transmission arrive at the destination, not just some of them. Sequence numbers are used on the sending end to allow the receiver's transport layer to identify and request missing segments.
- **Duplication Control:** It guarantees that no duplicate data arrives at the destination by using sequence numbers to identify and discard any accidental copies.

Q2: Explain TCP Congestion Control, including its different phases. (10 Marks / 10 Points)

TCP congestion control is a method used to manage data flow over a network and prevent congestion, acting as a "good neighbor" to avoid flooding the network and creating traffic jams for everyone else. It senses traffic on the entire network and adjusts the sending rate to avoid overwhelming the internet itself.

- **Distinction from Flow Control:** It is important not to confuse congestion control with flow control; flow control manages a one-to-one connection, while congestion control considers the bigger picture of the entire network.
- **Congestion Window (CWND):** TCP utilizes a congestion window (cwnd) and a congestion policy to manage and avoid network congestion. The network, in addition to the receiver, helps determine the appropriate size of the sender's window.
- **Slow Start Phase:** This phase begins cautiously by sending just one single segment of data and waiting for an acknowledgment. If successful, TCP doubles its sending window exponentially to two, then four, then eight segments, and so on, after each successful round trip.
- **Exponential Increment:** This exponential growth (e.g., $cwnd = 2^1$, then 2^2 , etc.) is a rapid way to figure out how much traffic the network can initially handle. This phase continues until the sending speed hits a predefined threshold (sssthresh).
- **Congestion Avoidance Phase:** Once the ascending speed hits the threshold, TCP switches from aggressive exponential growth to a much more careful, additive increment. In this phase, the CWND increases by only one single segment at a time after each Round Trip Time (RTT).
- **Additive Increment:** For example, if the CWND is 20 segments and all are acknowledged, it increases to 21 segments in the next RTT. This provides a gentler, more sustainable way to maximize speed without causing network breakdown.
- **Congestion Detection Phase:** If congestion occurs, indicated by the need to retransmit a segment, the congestion window size is decreased. Retransmission is usually triggered by an RTO (Retransmission Timeout) timer expiring or by receiving three duplicate ACKs.
- **Case 1:** Retransmission due to Timeout: If a timeout occurs, the possibility of high congestion is assumed. In this scenario, the sssthresh is reduced to half of the current window size, the cwnd is reset to 1, and TCP re-enters the slow start phase.
- **Case 2:** Retransmission due to 3 Duplicate ACKs: When three duplicate ACKs are received, it implies that congestion is less severe. Here, the sssthresh value reduces to half of the current window size, cwnd is set to sssthresh, and TCP immediately moves to the congestion avoidance phase.

Q3: Describe the TCP 3-way Handshake process for connection establishment. (5 Marks / 6 Points)

- **Connection-Oriented Protocol:** TCP is a connection-oriented protocol, which means it must establish a secure, logical connection (a virtual circuit) between the sender and receiver before any data can be transmitted. This process reserves resources at both ends.
- **Polite Introduction:** The TCP 3-way handshake is an elegant, three-step process that allows a client and server to politely introduce themselves and set up a solid, two-way communication channel.
- **Step 1 (SYN):** The client computer (sender) initiates the process by sending a SYN packet (synchronization segment). This packet includes a randomly generated initial sequence number

(e.g., Seq=521) and has the SYN flag set (Syn=1), essentially asking, "Hey server, are you there? Can you hear me?".

- **Step 2 (SYN/ACK):** The server receives the SYN packet and responds with a SYN/ACK packet. This response includes its own random initial sequence number (e.g., Seq=2000), a SYN flag (Syn=1), an ACK flag (ACK=1), and an acknowledgment number (e.g., Ack no.=522) confirming receipt of the client's SYN. This means, "Yep, I hear you loud and clear. Can you hear me?".
- **Step 3 (ACK):** Finally, the client sends one last ACK packet. This packet contains its next expected sequence number (e.g., Seq=522) and an acknowledgment number (e.g., Ack no.=2001) for the server's SYN, with the ACK flag set (ACK=1). This signals, "Roger that. I hear you, too. Okay, let's talk".
- **Connection Established:** After this successful three-step exchange, a solid, full-duplex, two-way connection is fully established and ready for data transmission, with both parties having synchronized their sequence numbers and agreed on window sizes.

Q4: Compare and contrast TCP and UDP protocols, highlighting their main differences and use cases. (5 Marks / 6 Points)

- **Connection Type:** TCP (Transmission Control Protocol) is a **connection-oriented protocol**, requiring a virtual circuit to be established between sender and receiver before data transfer. UDP (User Datagram Protocol) is a connectionless transport protocol, meaning it sends data without prior connection setup.
- **Reliability:** TCP provides a **reliable byte stream service**, guaranteeing the delivery of data and retransmitting lost or damaged packets. UDP provides a simple asynchronous datagram service and offers no inherent reliability guarantees, focusing on speed over error correction.
- **Speed & Overhead:** **UDP is incredibly fast** and lightweight because it has no setup, error checking, or other overhead. TCP has more overhead due to its connection establishment, extensive error checking, and confirmation processes, making it slower than UDP.
- **Error Checking:** **TCP performs end-to-end error control and retransmits segments** that are not acknowledged within a specific period. UDP includes a checksum for error detection, but it does not perform retransmission or extensive error handling.
- **Data Structure:** **TCP transmits data as continuous byte streams**, assigning a unique sequence number to each byte for in-order delivery. UDP transmits data in the form of user datagrams (packets) in a non-sequential manner.
- **Use Cases:** **TCP is perfect for applications that absolutely require 100% correctness** and complete data delivery, such as emails, file downloads, and web browsing (HTTP). UDP is ideal for applications where speed is paramount and a few dropped pixels or glitches don't significantly impair functionality, such as live video streaming, online gaming, and VoIP.

Q5: What is TCP Flow Control and how does it work? (5 Marks / 6 Points)

TCP flow control is a critical mechanism designed to prevent the sender from overwhelming the receiver with too much data, ensuring efficient and reliable data delivery. It guarantees in-order and error-free data transfer by managing the rate at which data is sent.

- **Sliding Window Protocol:** TCP utilizes a sliding window protocol for flow control, which makes data transmission more efficient and directly controls the flow of data so the receiver is not overloaded. This protocol is byte-oriented.
- **Initial Handshake:** During the connection establishment (the 3-way handshake), the receiver sends its initial window size to the sender. This advertised window size indicates the number of bytes its buffer can currently hold.
- **Sender Transmission:** The sender then transmits data segments, ensuring it does not send more data than the receiver's advertised window size.
- **Dynamic Window Adjustment:** As the receiver processes the data and frees up buffer space, it dynamically adjusts and advertises a larger window size to the sender, allowing for faster transmission. Conversely, if the receiver's buffer fills up, it advertises a smaller window or even a zero window to slow down or temporarily halt the sender.
- **Acknowledgments (ACKs):** For each segment successfully received, the receiver sends an acknowledgment (ACK), which can also carry the updated window size. TCP also uses window scaling for high-bandwidth connections and a persist timer to ensure the connection stays alive even with a zero window.

Q6: What is the primary definition of the Transport Layer? (2 Marks / 2 Points)

- **Fourth Layer:** The Transport Layer is officially the fourth layer in the standard network model.
- **Logical Communication for Applications:** Its main job is to provide a logical, direct line of communication between specific applications running on different computers (end systems), rather than just between the computers themselves.

Q7: Where are transport layer protocols implemented? (2 Marks / 2 Points)

- **End Systems Only:** The transport layer protocols are implemented exclusively in the end systems. This means they reside within devices like your computer, smartphone, or server.
- **Not in Routers:** These protocols are not implemented in network routers. Routers primarily operate at lower network layers, forwarding packets between different networks.

Q8: Explain the two types of multiplexing provided by the transport layer. (2 Marks / 2 Points)

- **Upward Multiplexing:** This occurs when multiple transport layer connections utilize the same single network connection. It's used to improve cost-effectiveness by sending several transmissions bound for the same destination along the same path.
- **Downward Multiplexing:** This involves one transport layer connection using multiple network connections simultaneously. It allows the transport layer to split a connection across several paths to enhance throughput, particularly useful when networks have low or slow capacity.

Q9: What role does addressing play in the Transport Layer? (2 Marks / 2 Points)

- Application-Specific Delivery: The transport layer is crucial for delivering data generated by an application on one machine to the correct application on another machine.
- Port Numbers: It achieves this by providing user addresses, specified as a "station" or "port". These port numbers allow the transport layer protocols to know which specific upper-layer applications are communicating.

Q10: What is Weighted Fair Queuing (WFQ) and how does it manage bandwidth? (2 Marks / 2 Points)

- Dynamic Queue Creation: WFQ dynamically creates separate queues for different traffic flows, identifying them based on various header fields in packets such as Source/Destination IP address or TCP/UDP port.
- Priority-Based Bandwidth Allocation: It assigns bandwidth to these flows based on priority, with packets of higher priority being scheduled before lower-priority ones. If a queue becomes idle, its freed-up bandwidth is dynamically reallocated among the remaining active queues while preserving their original bandwidth ratios.

Q11 Describe the various fields within a TCP segment header and their functions.**(10 Marks / 10 Points)**

- Header Length & Size: A TCP segment header can range from 20 to 60 bytes in length. By default, it is 20 bytes long, with the remaining 40 bytes being used for optional fields.
- Source Port Address: This is a 16-bit field that holds the port number of the application program that is sending the data segment.
- Destination Port Address: Also a 16-bit field, it stores the port number of the application running on the host that is intended to receive the data segment.
- Sequence Number: A 32-bit field that contains the byte number of the first byte of data in that specific segment. This number is critical at the receiving end for reassembling messages that may arrive out of order.
- Acknowledgment Number: This 32-bit field stores the byte number that the recipient expects to receive next. It acts as a confirmation that all preceding bytes have been successfully received.
- Header Length (HLEN): This 4-bit field indicates the total length of the TCP header, specified in 4-byte words. Its value ranges from 5 (for a 20-byte header, $5 \times 4 = 20$) to 15 (for a 60-byte header, $15 \times 4 = 60$).
- Control Flags: These are six individual 1-bit control bits that are used to regulate flow control, the method of transfer, and the establishment, abortion, or termination of a connection.
- URG (Urgent) Flag: When set, it indicates that the urgent pointer field is legitimate and contains relevant information.
- ACK (Acknowledgment) Flag: When set, it signifies that the acknowledgment number field is valid and should be used, particularly in cumulative acknowledgment scenarios.
- SYN (Synchronization) Flag: Used in the connection establishment phase (the 3-way handshake) to synchronize sequence numbers between the client and server.
- FIN (Finish) Flag: Used to signal the termination of communication. Other flags include PSH (Push request) and RST (Restart the link).

- **Window Size:** This field specifies the sender TCP's window size in bytes, indicating how much data the sender is currently willing to accept.
- **Checksum:** This field contains a checksum for error control. Unlike UDP, the checksum is a required component for TCP to ensure data integrity.
- **Urgent Pointer:** This field, valid only if the URG flag is set, points to data within the segment that needs to be urgently processed and delivered to the receiving application as soon as possible.

Q12 What services does UDP provide and what are the key fields in a UDP segment, including the detailed checksum calculation? (5 Marks / 6 Points)

- **Connectionless Service:** UDP (User Datagram Protocol) provides a connectionless service, meaning it does not establish a dedicated, secure connection before transmitting data.
- **Faster Delivery:** It is known for its faster delivery of messages compared to TCP, as it prioritizes speed and the size of data over strict security and reliability.
- **Process-to-Process Communication:** UDP facilitates process-to-process communication, allowing specific applications running on different hosts to send messages to each other directly.
- **Source Port:** This 2-byte field is used to identify the port number of the application (source) that is sending the UDP packet.
- **Destination Port:** This 2-byte field is used to specify the port number of the application (destination) that is intended to receive the UDP packet.
- **Length:** A 16-bit field that indicates the total length of the UDP packet, including both the UDP header and the user data.
- **Checksum (Detailed):** This two-byte field is used for error detection. It is calculated as the 16-bit one's complement of the one's complement sum of three components: the UDP header, a pseudo-header (containing information from the IP header), and the user data. If needed, the data is padded with zero octets at the end to ensure it forms a multiple of two octets for the calculation.